Table des matières

Cours 05 — Performance, requêtes et transformation 1
Objectifs 1
Théorie 1
QCM — style examen (Cours 05) 5
Points clés à retenir 10

Cours 05 — Performance, requêtes et transformation

Module 5/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

Comment lire ce module — Le domaine 4 pèse 21 %, le deuxième plus lourd de l'examen. Il a deux visages : diagnostiquer et accélérer (lire un *Query Profile*, comprendre les caches, actionner les bons leviers) et transformer la donnée (fenêtrage, semi-structuré). Le réflexe de fond : Snowflake fait déjà l'essentiel (micro-partitions, pruning, optimiseur) ; votre travail, c'est de repérer le point de blocage et de tirer le bon levier, pas de tout régler à la main.

Objectifs

À la fin de ce module, vous saurez :

- lire un Query Profile et nommer les symptômes classiques : pruning, spilling, exploding joins, file d'attente ;
- distinguer les trois caches (résultats, données local, métadonnées) ;
- choisir un levier de performance : dimensionner, clés de clustering, vues matérialisées, search optimization, Query Acceleration Service ;
- savoir où observer l'usage : `ACCOUNT_USAGE` vs `INFORMATION_SCHEMA`, et `GET_QUERY_OPERATOR_STATS` ;
- transformer le structuré (fenêtrage) et le semi-structuré (`VARIANT`, `FLATTEN`).

Théorie

1. Le modèle mental de la performance

Une requête lente, ce n'est presque jamais « Snowflake est lent » : c'est un goulot précis. Les trois familles de causes :

- le calcul — l'entrepôt est trop petit (la requête déborde) ou trop sollicité (les requêtes font la queue) ;

© 2026 iAlexMG. Tous droits réservés.

- la lecture — on lit trop de données (mauvais pruning) ;
- la requête elle-même — une jointure qui explose, ou une transformation maladroite.

Le premier levier, vous le connaissez déjà (Cours 01, section 5) : grossir l'entrepôt (*scale up*) pour une requête lourde, multiplier (multi-cluster, *scale out*) pour la concurrence. Le reste de ce cours, c'est diagnostiquer avant d'agir.

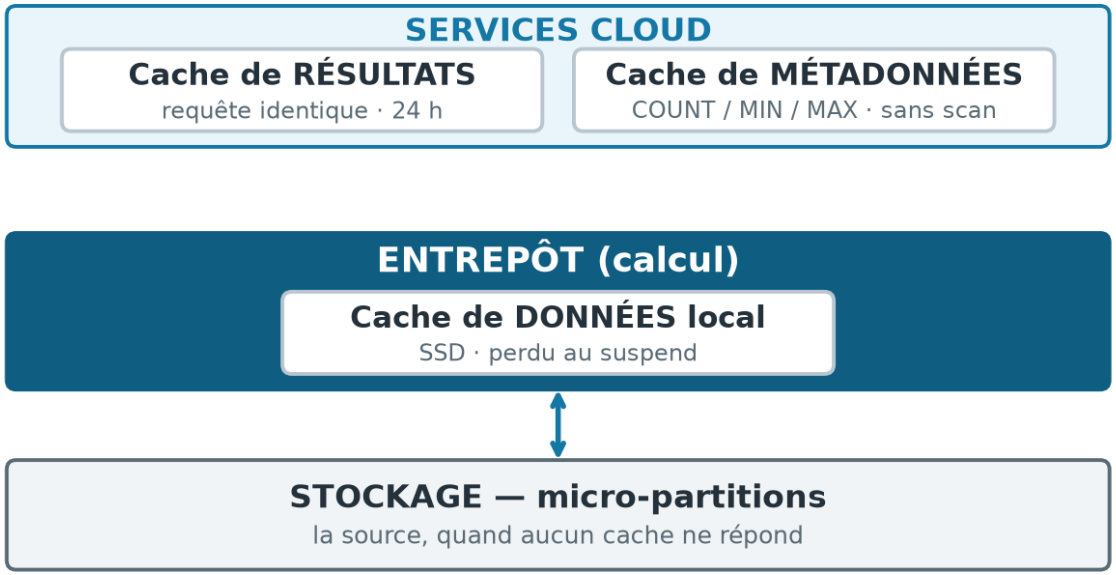
2. Lire un Query Profile

Labo :** `Labo_05_SNOW.md`**, étape 3.

Le Query Profile (Query History → une requête → onglet *Query Profile*) est l'arbre des opérateurs d'une requête, avec leurs statistiques. Quatre symptômes reviennent à l'examen :

- Pruning — sur un nœud `TableScan`, *partitions scanned* vs *total* (Cours 01, section 6). Peu élagué = on lit trop ; souvent un problème de rangement (clustering, section 4).
- Spilling (débordement) — l'entrepôt manque de mémoire et déverse sur le disque : *bytes spilled to local storage* (le SSD des nœuds, déjà un ralentissement), puis pire, *to remote storage*. Remède : grossir l'entrepôt (plus de mémoire par nœud).
- Exploding join (jointure qui explose) — une jointure produit beaucoup plus de lignes qu'elle n'en reçoit (condition trop lâche, doublons dans une clé). Le profil montre un `Join` dont la sortie dépasse de loin l'entrée.
- File d'attente (*queuing*) — la requête attend un entrepôt saturé (*queued overload time*). Remède : multiplier (multi-cluster) pour absorber la concurrence.

© 2026 iAlexMG. Tous droits réservés.



Les trois caches empilés : le cache de résultats et le cache de métadonnées dans les services cloud, le cache de données local dans l'entrepôt, et le stockage en dessous.

3. Les trois caches

Labo : `Labo_05_SNOW.md`, étapes 1 et 2.

Snowflake évite du travail grâce à trois caches distincts, un classique d'examen :

Cache	Où	Ce qu'il sert	Durée de vie
Cache de résultats	services cloud	le résultat exact d'une requête déjà exécutée	24 h (prolongé à chaque réutilisation, jusqu'à 31 j)
Cache de données local	l'entrepôt (SSD des nœuds)	les micro-partitions déjà lues	tant que l'entrepôt tourne (perdu au *suspend*)
Cache de métadonnées	services cloud	des réponses sans scan (`COUNT(*)`, `MIN`/`MAX`…)	permanent (métadonnées des micro-partitions)

- Le cache de résultats ressert une requête identique si les données n'ont pas changé, sans rallumer l'entrepôt (vous l'avez vu au Labo 01). On peut le désactiver le temps d'un test : `ALTER SESSION SET USE_CACHED_RESULT = FALSE;`.

© 2026 iAlexMG. Tous droits réservés.

- Le cache local explique pourquoi une 2e requête (différente mais sur les mêmes données) va plus vite : les partitions sont déjà en SSD. Suspendre l'entrepôt le vide.
- Le cache de métadonnées répond à `SELECT COUNT(*) FROM t;` instantanément, sans entrepôt : le compte est déjà dans les métadonnées.

4. Les leviers de performance

Labo : `Labo_05_SNOW.md`, étape 6.

Quand le diagnostic est posé, on choisit un levier :

- Dimensionner — grossir (requête lourde, spilling) ou multiplier (concurrence, file d'attente). Le levier numéro un, souvent suffisant.
- Clés de clustering — sur de très grandes tables, `CLUSTER BY (colonne)` regroupe physiquement les lignes souvent filtrées ensemble → meilleur pruning. Le service Automatic Clustering l'entretient (coût serverless). Inutile à petite échelle.
- Vues matérialisées (Enterprise+) — stockent un résultat précalculé et le maintiennent à jour ; utiles pour une agrégation coûteuse relue souvent (avec des restrictions).
- Search Optimization Service (Enterprise+) — accélère les recherches très sélectives (« trouver une aiguille » : `WHERE id = …`, égalité sur une colonne à forte cardinalité). `ALTER TABLE … ADD SEARCH OPTIMIZATION;`.
- Query Acceleration Service, ou QAS (Enterprise+) — prête du compute serverless aux requêtes dont le scan est gros et variable, sans grossir l'entrepôt en permanence. `ALTER WAREHOUSE … SET ENABLE_QUERY_ACCELERATION = TRUE;`.

Le bon levier pour le bon mal — *Spilling* → grossir. *File d'attente* → multiplier. *Mauvais pruning* → clé de clustering. *Point-lookup* sélectif → search optimization. *Agrégation relue souvent* → vue matérialisée. *Scan gros et variable* → QAS. L'examen adore ces correspondances.

5. Où observer : `ACCOUNT_USAGE` vs `INFORMATION_SCHEMA`

Labo : `Labo_05_SNOW.md`, étape 4.

Deux sources de métadonnées d'usage, à ne pas confondre :

- `SNOWFLAKE.ACCOUNT_USAGE` — un schéma de la base système `SNOWFLAKE` : historique long (jusqu'à 365 jours), à l'échelle du compte. Mais latence (jusqu'à ~45 min à 3 h) et accès réservé (rôle avec *imported privileges* ; ACCOUNTADMIN par défaut). Pour l'audit et les tendances. Ex. : `SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY` (avec `PARTITIONS_SCANNED`, `BYTES_SPILLED_TO_LOCAL_STORAGE`…).
- `INFORMATION_SCHEMA` — un schéma par base, en temps réel, rétention courte (7–14 j), exposé par des fonctions de table : `TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())`. Pour le dernier état, tout de suite.
- `GET_QUERY_OPERATOR_STATS(query_id)` — le Query Profile en SQL : les statistiques par opérateur (dont le pruning), immédiatement (vous l'avez utilisé au Labo 01).

© 2026 iAlexMG. Tous droits réservés.

Le piège d'examen — *Account Usage* = long historique, mais différé et privilégié. *Information Schema* = temps réel, mais court et par base. Si la question dit « il y a une latence » ou « un an d'historique », c'est `ACCOUNT_USAGE`.

6. Transformer la donnée

Labo : `Labo_05_SNOW.md`, étape 5.

a) Le structuré, le fenêtrage. Une fonction de fenêtre (*window function*) calcule sur un groupe de lignes voisines sans réduire le résultat (contrairement à `GROUP BY`) :

```
SELECT nom, region, salaire,
       RANK() OVER (PARTITION BY region ORDER BY salaire DESC) AS rang_regional
FROM   employes;
```

Ici, chaque ligne garde son détail et reçoit son rang dans sa région. `ROW_NUMBER`, `RANK`, `SUM(...) OVER (...)`, `LAG`/`LEAD` sont les grands classiques. Les CTE (`WITH … AS (…)`) rendent ces requêtes lisibles en les découpant en étapes nommées.

b) Le semi-structuré, `VARIANT` et `FLATTEN`. Le JSON vit dans une colonne `VARIANT` (Cours 01, section 7). On navigue avec `:` (champ) et `[ ]` (élément), on type avec `::` :

```
SELECT charge:type::string AS type,
       charge:tags[0]::string AS premier_tag
FROM   evenements;
```

Et pour éclater un tableau JSON en une ligne par élément, on utilise `LATERAL FLATTEN` :

```
SELECT e.id, f.value::string AS tag
FROM   evenements e,
       LATERAL FLATTEN(input => e.charge:tags) f;   -- 1 ligne par tag
```

C'est le cœur de la transformation semi-structurée : aplatir un document en table exploitable.

QCM — style examen (Cours 05)

Comment répondre — Faites les dix questions avant d'ouvrir le corrigé, qui commence à la page suivante. Une question à choix multiple dit combien de réponses cocher, comme à l'examen. Comptez une minute par question.

1. Le profil d'une requête affiche des octets déversés vers le stockage local, puis vers le stockage distant. Elle est seule à tourner sur son entrepôt.
    a. Poser une clé de clustering sur la colonne filtrée.
    b. Passer l'entrepôt en multi-cluster, pour que les clusters se partagent la charge.
    c. Vider le cache de résultats, qui occupe la mémoire des nœuds.

© 2026 iAlexMG. Tous droits réservés.

d. Grossir l’entrepôt, chaque cran ajoutant de la mémoire par nœud.

2. Le lundi matin, vingt analystes lancent leurs rapports en même temps. Le profil montre un long temps d’attente en file, et les requêtes elles-mêmes sont rapides une fois parties.
    a. Grossir l’entrepôt d’un cran.
    b. Passer l’entrepôt en multi-cluster, avec sa politique de mise à l’échelle.
    c. Poser une vue matérialisée derrière chaque rapport.
    d. Activer le Search Optimization Service sur les tables lues.
3. Un `TableScan` annonce 4 000 partitions lues sur 4 000, alors que la requête ne demande qu’une seule journée sur trois ans d’historique.
    a. La requête passe par une vue sécurisée, qui interdit l’élagage.
    b. C’est normal : l’élagage ne joue que sur les jointures.
    c. La table n’est pas rangée selon la colonne filtrée, et une clé de clustering rapprocherait les lignes d’une même journée.
    d. L’entrepôt est trop petit pour élaguer.
4. *(choix multiple, sélectionnez-en DEUX)* Deux de ces affirmations sur les caches sont exactes.
    a. Le cache de résultats ressert une requête identique sans rallumer l’entrepôt, tant que les données n’ont pas changé.
    b. Le cache de données local survit à la suspension de l’entrepôt.
    c. `SELECT COUNT(*) FROM t` se lit dans le cache de métadonnées, sans lire les données.
    d. Le cache de résultats se vide dès qu’on change de rôle.
5. Une application cherche un client par son identifiant, des milliers de fois par jour, dans une table de quatre milliards de lignes.
    a. Le Search Optimization Service, fait pour les recherches très sélectives.
    b. Une vue matérialisée sur la table entière.
    c. Le Query Acceleration Service, qui prête du calcul serverless.
    d. Un entrepôt de deux crans plus gros, pour lire la table plus vite.
6. Un tableau de bord rejoue la même agrégation coûteuse toutes les heures, sur des données qui ne changent que la nuit.
    a. Une clé de clustering sur la colonne agrégée.

© 2026 iAlexMG. Tous droits réservés.

b. Le Search Optimization Service sur la table de faits.

c. Une vue matérialisée, qui stocke le résultat et se maintient seule.

d. Le cache de résultats, qu'il suffit d'activer pour la session du tableau de bord.

7. On vous demande la dépense de crédits par entrepôt sur les douze derniers mois, et une demi-heure de retard sur les chiffres ne dérange personne.

   a. `INFORMATION_SCHEMA.QUERY_HISTORY()`, qui répond en temps réel.

   b. `SHOW WAREHOUSES`, qui porte l'historique de chaque entrepôt.

   c. Le cache de métadonnées, où vivent les compteurs du compte.

   d. `SNOWFLAKE.ACCOUNT_USAGE`, qui garde jusqu'à 365 jours, avec une latence.

8. *(choix multiple, sélectionnez-en DEUX)* Vous voulez savoir, en SQL, combien de micro-partitions une requête a réellement lues. Deux de ces affirmations sont exactes.

   a. `INFORMATION_SCHEMA.QUERY_HISTORY()` porte les colonnes `PARTITIONS_SCANNED` et `PARTITIONS_TOTAL`.

   b. `GET_QUERY_OPERATOR_STATS` rend les statistiques par opérateur, dont l'élagage, sans attendre.

   c. Seul le Query Profile de Snowsight donne l'information : aucune fonction SQL ne la rend.

   d. `SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY` porte `PARTITIONS_SCANNED` et les octets déversés, avec sa latence habituelle.

9. Pour chaque vente, on veut son rang dans sa région, et la liste doit garder une ligne par vente.

   a. Un `GROUP BY region` avec un `MAX(montant)`.

   b. `RANK() OVER (PARTITION BY region ORDER BY montant DESC)`.

   c. Un `LATERAL FLATTEN` sur la colonne de région.

   d. Une vue matérialisée par région, triée par montant.

10. Une colonne `VARIANT` porte, pour chaque événement, un tableau d'étiquettes. On veut une ligne par étiquette.

    a. `LATERAL FLATTEN(input => charge:tags)`, qui éclate le tableau en lignes.

    b. `charge:tags::string`, qui rend chaque étiquette sur sa propre ligne.

    c. Un `GROUP BY` sur la colonne `VARIANT`, qui sépare les éléments.

    d. `PARSE_JSON` suivi d'un `ORDER BY` sur le tableau.

© 2026 iAlexMG. Tous droits réservés.

© 2026 iAlexMG. Tous droits réservés.

Corrigé expliqué — Cours 05

À lire après avoir répondu. Chaque réponse nomme la section à relire : ce que vous ratez ici vous dit quoi reprendre.

1. Réponse d. Le débordement (*spilling*) dit une chose : l'entrepôt manque de mémoire. Le remède direct est de grossir, puisque chaque cran double les nœuds et la mémoire. Le multi-cluster de l'option b sert la concurrence, et la requête est seule. La clé de clustering de l'option a réduit les lectures, pas le besoin de mémoire d'un tri ou d'une jointure. L'option c invente un cache qui occuperait la mémoire de calcul (sections 2 et 4).

2. Réponse b. Une file d'attente est un problème de concurrence : on multiplie les clusters, et la politique de mise à l'échelle décide quand en ouvrir un de plus. Grossir, à l'option a, accélérerait une requête déjà rapide. Les options c et d s'attaquent au contenu des requêtes, alors que le temps perdu l'est avant même qu'elles partent (sections 2 et 4).

3. Réponse c. Lire toutes les partitions pour une seule journée, c'est un élagage manqué : les lignes de cette journée sont dispersées dans la table, et une clé de clustering les rapproche. L'option a prête à la vue sécurisée un effet qu'elle n'a pas sur le pruning des partitions. L'option b se trompe de mécanisme. L'option d confond la taille de l'entrepôt avec la quantité de données à lire (sections 2 et 4).

4. Réponses a et c. Le cache de résultats ressert une requête identique sans entrepôt, tant que les données n'ont pas bougé, et le cache de métadonnées répond à un `COUNT(*)` sans rien lire. L'option b se trompe sur le cache local, qui vit dans le SSD des nœuds et disparaît à la suspension. L'option d invente un lien entre le cache et le rôle courant (section 3).

5. Réponse a. Chercher une aiguille par égalité sur une colonne à forte cardinalité, c'est le cas du Search Optimization Service. Une vue matérialisée, à l'option b, sert une agrégation relue, pas un point d'accès unitaire. Le Query Acceleration Service de l'option c vise les scans gros et variables. Grossir, à l'option d, ferait payer un plus gros entrepôt pour lire ce qu'il faudrait justement éviter de lire (section 4).

6. Réponse c. Une vue matérialisée stocke le résultat précalculé et se maintient seule : c'est le levier de l'agrégation coûteuse relue souvent. L'option d oublie que le cache de résultats est déjà là, et qu'il tombe dès que les données changent, c'est-à-dire chaque nuit. Les options a et b visent la lecture de lignes, pas le recalcul d'une agrégation (sections 3 et 4).

7. Réponse d. Un an d'historique à l'échelle du compte, avec de la latence, c'est la signature d'`ACCOUNT_USAGE`. `INFORMATION_SCHEMA`, à l'option a, répond tout de suite mais ne garde qu'une à deux semaines et ne voit qu'une base. `SHOW WAREHOUSES` de l'option b décrit les entrepôts, sans historique de dépense. L'option c prête au cache de métadonnées un rôle de comptabilité (section 5).

8. Réponses b et d. `GET_QUERY_OPERATOR_STATS` donne le Query Profile en SQL, immédiatement, et `ACCOUNT_USAGE.QUERY_HISTORY` porte `PARTITIONS_SCANNED` et les octets déversés, avec sa latence. L'option a est le piège : la fonction de table d'`INFORMATION_SCHEMA` n'a pas ces colonnes-là. L'option c oublie la fonction, que le Labo 01 vous a pourtant fait lancer (section 5).

9. Réponse b. Une fonction de fenêtre calcule sur un groupe voisin sans replier les lignes : le détail reste, le rang s'ajoute. Le `GROUP BY` de l'option a rend une ligne par région, ce que l'énoncé refuse. `FLATTEN`, à l'option c, sert le semi-structuré. L'option d fabrique des objets pour une question qui se règle dans la requête (section 6).

© 2026 iAlexMG. Tous droits réservés.

10. Réponse a. `LATERAL FLATTEN` éclate un tableau semi-structuré en une ligne par élément : c'est le cœur de l'aplatissement. L'option b type un tableau entier en texte, sur une seule ligne. Un `GROUP BY` ne sépare rien, contre l'option c, et `PARSE_JSON` de l'option d convertit du texte en `VARIANT` sans l'éclater (section 6).

Points clés à retenir

- Diagnostiquer d'abord : le Query Profile montre le pruning (partitions scanned/total), le spilling (mémoire → disque), les exploding joins (sortie >> entrée) et la file d'attente (*queued overload*).
- Trois caches : résultats (services cloud, 24 h, requête identique) ; données local (SSD de l'entrepôt, perdu au *suspend*) ; métadonnées (`COUNT`/`MIN`/`MAX` sans scan).
- Le bon levier : *spilling* → grossir ; *file d'attente* → multiplier ; *mauvais pruning* → clé de clustering ; *point-lookup* → search optimization ; *agrégation relue* → vue matérialisée (Enterprise+) ; *scan gros et variable* → QAS (Enterprise+).
- Observer : `ACCOUNT_USAGE` (base `SNOWFLAKE`, 365 j, différé, privilégié) vs `INFORMATION_SCHEMA` (temps réel, court, par base) ; `GET_QUERY_OPERATOR_STATS` = le profil en SQL.
- Transformer : fenêtrage (`RANK`/`ROW_NUMBER`/`SUM … OVER`, garde le détail) + CTE pour la lisibilité ; semi-structuré via `VARIANT` (`:` / `[ ]` / `::`) et `LATERAL FLATTEN` (éclater un tableau en lignes).

© 2026 iAlexMG. Tous droits réservés.